

Pepquiz - in-class 30 minutes + discussion + self-grading

1. Your source file `main.cpp` is located in the folder `/home/src/`. Inside that same project you have created your own header file named `Rectangle.h`, which is located in `/home/include/`. Write two different valid `#include` statements that will correctly include `Rectangle.h` from inside `main.cpp` - first using an **absolute**, second using a **relative** path to the header file.

```
#include "/home/include/Rectangle.h" // absolute path
#include "../include/Rectangle.h"    // relative path
```

2. Write a statement that defines an empty vector object named `avect` that can hold strings. Show that the vector is actually empty.

```
vector<string> avect;
cout << boolalpha << "The vector is empty: " << avect.empty();
cout << "\nThe vector is empty: size() = " << avect.size();
```

```
The vector is empty: true
The vector is empty: size() = 0
```

3. Write a statement that defines a vector object named `avect` that can hold integers. The vector should have ten elements initialized with the default value 0. Display the elements to show success.

```
vector<int> avect(10,0);
for (const auto& element : avect) cout << element << " ";
```

```
0 0 0 0 0 0 0 0 0 0
```

4. Write a statement that defines a vector object named `avect` that can hold integers. The vector should have 100 elements, each initialized with the value 1. If this worked, print a success message, but if any value is not 1, abort.

```
vector<int> avect(100,1);
for (const auto& element : avect)
    if (element != 1) cout << "Error!" << endl;
cout << "Initialized to 1!" << endl;
```

```
Initialized to 1!
```

You can test the program by changing one value of the vector:

```
vector<int> avect(100,1);
//int zero = 0;
//swap(avect[2],zero);
for (const auto& element : avect)
    if (element != 1) return EXIT_FAILURE;
cout << "Initialized to 1!" << endl;
```

```
Initialized to 1!
```

5. Write a statement that defines a vector object named v1 that can hold integers. The vector should be a copy of another vector v2. Display both vectors to show that the copying worked.

```
vector<int> v2 {1,2,3};
vector<int> v1(v2);
for (const auto& element: v1) cout << element << " ";
cout << endl;
for (const auto& element: v2) cout << element << " ";
```

```
1 2 3
1 2 3
```

6. What happens when you use an invalid index with the at function of the vector class? Try it.

```
vector<string> names {"Jim", "Jill", "Jack"};
cout << names.at(0); // valid index
cout << names.at(3); // invalid index
```

Answer: out_of_range exception is thrown. This happens at run-time.

terminate called after throwing an instance of 'std::out_of_range' what(): vector::Mrangecheck: —n (which is 3) >= this->size() (which is 3)

But: If it crashes at runtime, why does the first cout not execute?

It does execute but the output buffer is not flushed automatically here (in Emacs). To stop the program from dying, do this:

```
vector<string> names {"Jim", "Jill", "Jack"};

cout << names.at(0);           // prints Jim

try {
    cout << names.at(3);       // throws here
}
catch (const out_of_range& e) {
    cout << "\nError: " << e.what() << endl;
}
```

7. If your program will add a lot of objects to a vector, is it best to use the insert member function, or the emplace member function, and why?

Using the insert (and the push_back) member function may create copies of objects when inserting them into the vector, especially if you pass in fully constructed objects. In contrast, emplace (and emplace_back) constructs the object in-place, directly in the vector's memory, avoiding unnecessary copying or moving.

This is typically more efficient, especially when dealing with complex objects. Therefore, emplace is preferred when adding many objects, particularly if they can be constructed directly with constructor arguments.

8. Internally, how does a vector store its elements?

A vector stores its elements in a contiguous block of memory on the heap (the "free store") just like a dynamically allocated C-style array. It manages memory automatically and resizes as needed by allocating a larger array and copying existing elements when capacity is exceeded.

9. The vector class has a size and a capacity member function. What is the difference between these two member functions?

The size function returns the current size of the vector (in number of elements) while the capacity function returns the number of elements that can be stored before resizing is needed. It can be changed with the reserve function.

10. What is the difference between the vector class member functions insert and push_back? Show it with a short program.

- push_back adds a single element to the end of the vector. Syntax:
vector.push_back(value);
- insert adds one or more elements at a specific position in the vector. Syntax:
vector.insert(position, count, value);

Code example:

```
// Define an empty vector of integers.
vector<int> v;
// Add five numbers 1,2,3,4,5 to the vector.
for(int i=0;i<5;i++) v.push_back(i+1);
// Display vector content.
for(const auto& elem : v) cout << elem << " ";
// Create an iterator pointing at the position before 4
auto it = v.begin() + 2;
// Show the iterator value.
cout << "\nIterator positioned before: " << v.at(*it);
// Insert two numbers using the iterator.
v.insert(it, {40,50});
// Display vector content.
cout << endl;
for(const auto& elem : v) cout << elem << " ";
```

```
1 2 3 4 5
Iterator positioned before: 4
1 2 40 50 3 4 5
```

Careful: push_back dynamically changes the underlying array. As a result, iterators are invalidated (they're now dangling, because the memory has been rejigged). So this leads to disaster:

```
vector<int> v {0}; // vector has one element v[0]=0
for (auto it = v.begin(); it != v.end(); ++it) // iterator points at end
{
    v.push_back(*it); // modifies the vector while iterating over it
}
```

Author: marcus

Created: 2025-11-18 Tue 09:55